DESIGN AND ANALYSIS OF ALGORITHMS · DAA LAB

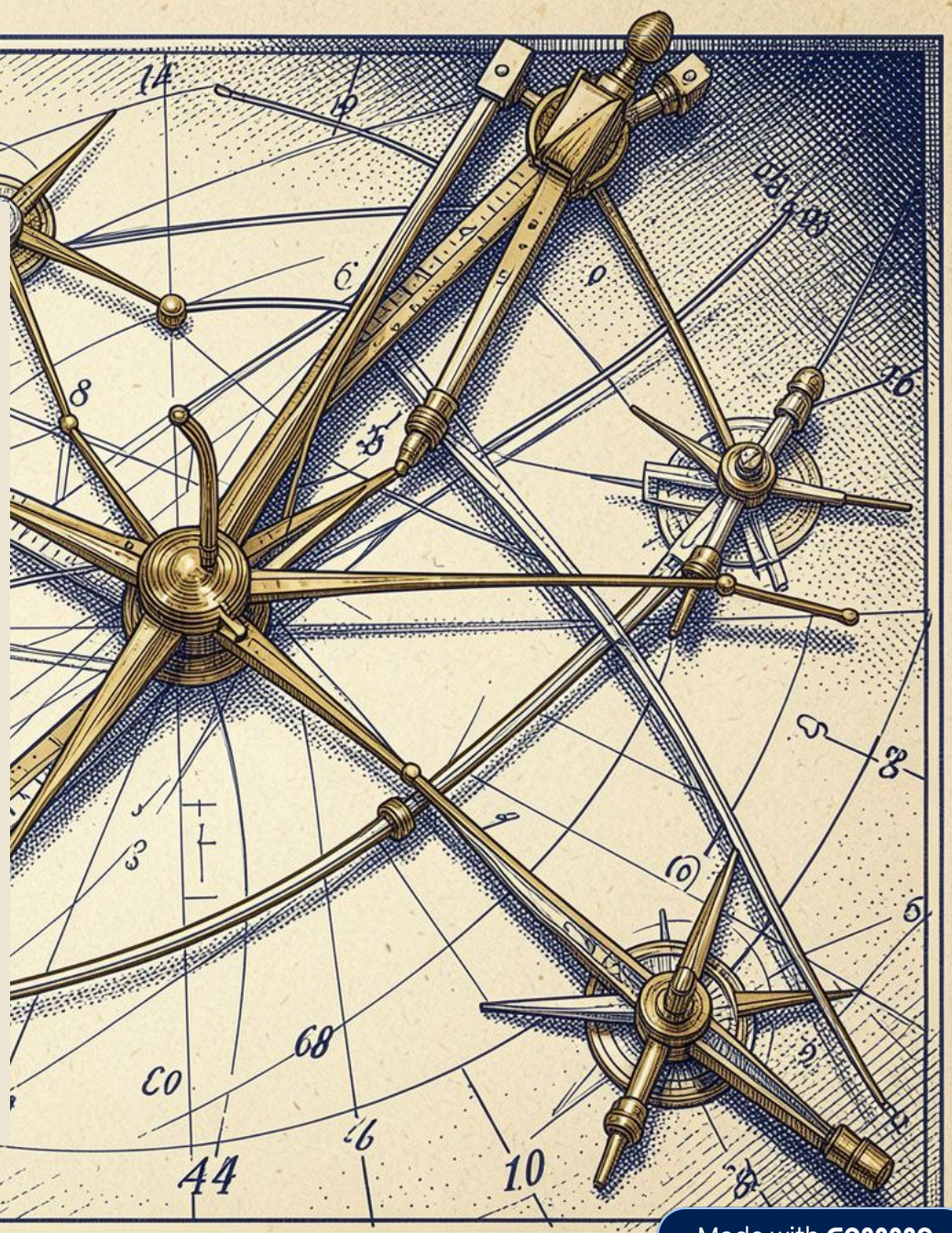
Dijkstra's Shortest Path Algorithm

Greedy Algorithms

Student name: Chiranshu Somkuwar PRN: 25070521213

Sem/Section: III/ D

Generated using Gamma AI



Made with GAMMA

THE ROUTING QUESTION

Shortest paths answer a routing question

Which route costs least from one source to every reachable vertex?



Weighted cost

Distance, time, energy, or risk.



Non-negative edges

Dijkstra assumes every edge weight is at least zero.



Useful output

Shortest distances plus predecessor links for reconstruction.

Made with GAMMA

THE GREEDY RULE

Dijkstra makes one safe greedy choice at a time

At each step, select the unvisited vertex with the smallest tentative distance and finalize it.

01

Pick

Choose the smallest tentative distance.

02

Relax

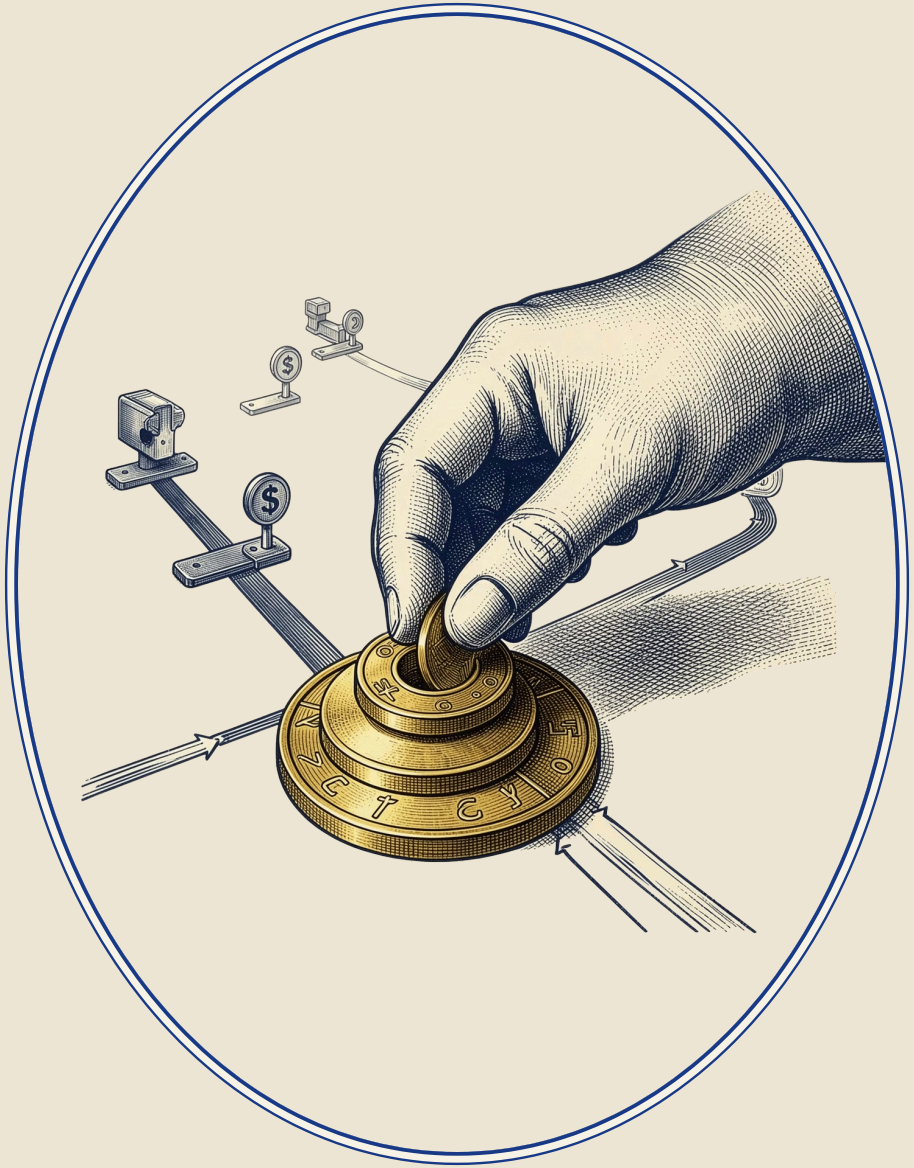
Improve neighbors when a shorter route appears.

03

Finalize

Mark the vertex safe and repeat.

Why it works: with non-negative weights, no later route can make the smallest tentative distance smaller.



Choose the closest unsettled vertex first.

Made with GAMMA

ALGORITHM

Initialize, relax, finalize

01

Initialize

Set source distance to 0; all others to ∞.

02

Choose

Pick the minimum-distance unvisited vertex.

03

Relax

Update a neighbor and store its predecessor.

04

Finalize

Mark the vertex visited.

05

Stop?

Target finalized, or no vertex remains.

Pseudocode

dist[s] = 0; all other distances = ∞

while unvisited:

u = argmin dist[u]

for each (u, v, w):

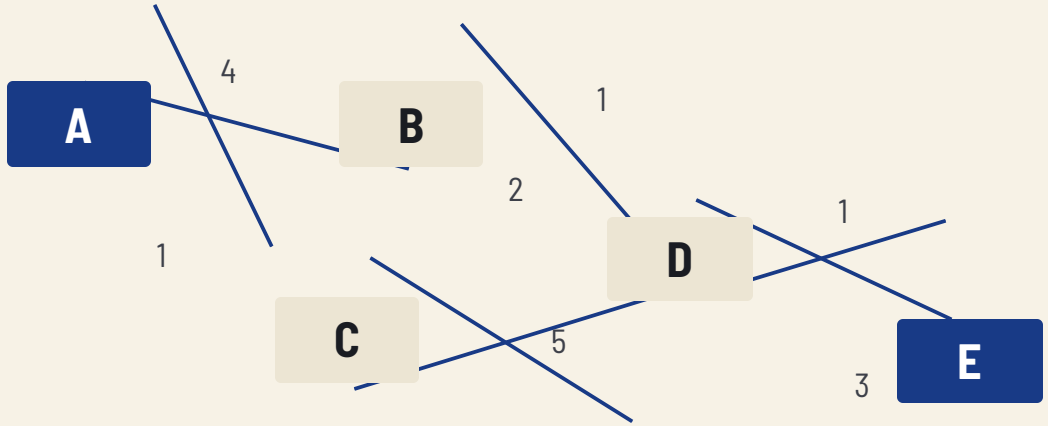
if dist[u] + w < dist[v], update dist[v] and prev[v]

Flow: Start → Initialize → Pick minimum → Relax neighbors → Finalize → More vertices? → End

Made with GAMMA

WORKED EXAMPLE

A to E costs 7



Final path: A → C → B → D → E

DISTANCE UPDATES

Finalized	d(A)	d(B)	d(C)	d(D)	d(E)
A	0	4	1	∞	∞
C	0	3	1	6	∞
B	0	3	1	4	10
D	0	3	1	4	7
E	0	3	1	4	7

1 + 2 + 1 + 3 = **7**

Made with GAMMA

IMPLEMENTATION CHOICES

Complexity depends on the priority queue

Representation	Selection structure	Time complexity
Adjacency list	Binary heap	**$O((V + E) \log V)$**, commonly $O(E \log V)$
Adjacency matrix	Linear scan	**$O(V^2)$**
Space	List + distances + predecessors + queue	$O(V + E)$

Validity condition

All edge weights must be non-negative.

Negative edges?

Use another algorithm, such as Bellman–Ford.

Made with GAMMA

FROM GRAPH TO SYSTEM

Shortest paths power real systems



GPS and maps

Minimize travel distance or time.



Networks

Choose low-cost paths between routers.



Robotics and games

Navigate a weighted movement graph.



Telecom and logistics

Optimize routes as costs change.

Each application turns a network into vertices, edges, and non-negative weights.

Made with GAMMA

TAKEAWAY

Greedy choice. Verified path. Practical impact.

A simple rule becomes an efficient shortest-path engine.

What to remember

- Finalize the closest unsettled vertex.
- Relaxation updates routes and predecessors.
- Correctness requires non-negative edge weights.
- Choose the implementation for graph density and priority-queue needs.

Made with GAMMA